BERND BLINDOW GRUPPE
SCHULE AUSBILDUNG STUDIUM

SCHULPROJEKT / C++17 / 2026-2027

Anforderungs-analyse

VIER GEWINNT · GYM EDITION

Fachliche Anforderungen, Erweiterungen, Abnahmetests und Rückverfolgbarkeit. Grundlage für eine objektorientierte C++-Implementierung.

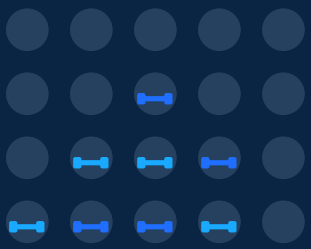
ZUM INTERAKTIVEN INHALTSVERZEICHNIS →

DOKUMENTENSTAND

Arbeitsfassung 1.0 · 22.10.2026

Julian Krauß | ITA-09/25

Bernd-Blindow-Schulen · Ravensburg



DESIGN-ID
JK-CD-DE-2026-003

WERK-ID
JK-PT-2026-051-RA

© 2026 Julian Krauß · Analyse- und Planungsdokument

INDEX / NAVIGATION

Inhaltsverzeichnis

Klicke auf eine Zeile, um direkt zum Abschnitt zu springen. Auf jeder Folgeseite führt der Kopfbereich zurück zum Index.

01	Entscheidungsvorlage & Projektziel	S. 3
02	Projektgrenzen und Rollen	S. 4
03	Pflichtanforderungen I	S. 5
04	Pflichtanforderungen II	S. 6
05	Pflichtanforderungen III	S. 7
06	Geplante Sonderfunktionen	S. 8
07	Gaming-UI und Konsole	S. 9
08	Nichtfunktionale Anforderungen	S. 10
09	Architektur und Klassen	S. 11
10	Speichern, Laden, Highscore	S. 12
11	Testkonzept: Kernabläufe	S. 13
12	Testkonzept: Daten und Extras	S. 14
13	Traceability: Pflichtfunktionen	S. 15
14	Traceability: Zusatzumfang	S. 16
15	Priorisierung und Termine	S. 17
16	Risiken, Entscheidungen, Fragen	S. 18
17	Abnahmecheckliste & Quellen	S. 19

LEGENDE DER KENNUNGEN

FR = verbindliche fachliche Anforderung aus Aufgabenblatt 51; **EX** = projektinterne Zusatzfunktion; **NFR** = Qualitätsanforderung; **TC** = Testfall; **P** = Programmablaufplan (PAP).

01 / ÜBERBLICK

Projektziel und Ergebnis

Eine prüfbare Ausgangsbasis für Entwicklung, Zwischenstände, Tests und die Abschlussvorführung am 18.02.2027.

PROJEKTZIEL

Entwickelt wird ein objektorientiertes Konsolenspiel „Vier gewinnt“ in C++ für **zwei bis vier menschliche Spieler**. Vor jeder Partie werden Spielernamen und Spielerzahl erfasst; das Spielfeld wächst mit der Spielerzahl. Gewinn, Remis, Eingabefehler, Speichern, Laden und Highscore müssen zuverlässig funktionieren.

Pflichtumfang

Hauptmenü, Spielerverwaltung, skalierbares Spielfeld, Regelprüfung, Textdateien, Highscore, GitHub, Handout, Präsentation und KI-Prompt-Dokumentation.

Geplanter Zusatzumfang

Kurzhantel-Symbole, kleine Fallanimation, Vorschau, Rückgängig, Hilfe, Spielstatistik und schneller Speicherbefehl. Optionale Gaming-UI nur als Entwurf.

Außerhalb der Pflicht

Kein Online-Multiplayer, keine KI-Gegner, kein Benutzerkonto und keine echte 3D-Game-Engine. Ein grafisches Spiel ist von Aufgabenblatt 51 nicht gefordert.

Vorrang der Stabilität

Die Programmfunktion zählt laut Bewertungsmatrix 60 %, Präsentation und Handout jeweils 20 %. Deshalb kommen Zusatzfeatures erst nach den Pflichtfunktionen.

Eindeutige Abgrenzung des Umsetzungsstands

- **Nachweisbar vorhanden:** Ein C++17-Konsolen-Lernprojekt samt Header-, CPP-Dateien und Dokumentation wurde als Arbeitsstand erstellt.
- **Noch nicht als abgeschlossen belegt:** Individuelle Schulabnahme, vollständige Prüfergebnisse, GitHub-Commit-Historie und die produktive grafische Gaming-Oberfläche.

AKZEPTANZKRITERIUM AUF PROJEKTEBENE

Alle Muss-Anforderungen erfüllen ihre benannten Tests; der Entwickler erläutert die Klassen und Regeln und führt das Programm mit nachvollziehbarer Historie live vor.

© 2026 Julian Krauß | JK-PT-2026-051-RA

02 / ANALYSEBASIS

Projektgrenzen und Rollen

Die Anforderungsanalyse bezieht sich auf Aufgabenblatt 51; projektinterne Designentscheidungen sind ausdrücklich getrennt.

Beteiligte und Erwartungen

Rolle	Erwartung / Verantwortung
Auftraggeber / Fachlehrkraft	Funktionsfähige Konsolenanwendung, dokumentierte Eigenleistung, OOP und nachvollziehbare Vorführung.
Entwickler / Projektteam	Geordnete Entwicklung in C++17, Versionsverwaltung, Tests, Dokumentation und begründete Entscheidungen.
Spielende (2-4 Personen)	Schnelle Bedienung, verständliche Regeln, sichtbarer Spielstatus und robuste Fehlermeldungen.
Präsentationspublikum	Klare Demonstration mit verständlicher Erklärung von Architektur, Gewinnprüfung und Dateiformat.

Nutzungskontext und Systemgrenze

- **Zielplattform:** Desktop-Konsole bzw. Terminal. Compiler nach C++17-Standard, lokale Speicherung ohne Cloud-Zwang.
- **Spiellogik:** Bis zu vier menschliche Spieler; alle benutzen dasselbe Gerät. Keine automatische Spielfigur.
- **Persistenz:** Lokale Dateien für aktuellen Spielstand und Siegstatistik; beschädigte Dateien dürfen keinen Absturz auslösen.
- **Designtrennung:** Kurzhantel-Darstellung ändert nur die Optik, nicht die Gewinnregeln. Die gezeigten 3D-Bilder sind unverbindliche UI-Mockups.

FACHLICHE ANNAHME

Die Spielfeldbreite lautet **7 + 2 × (Spielerzahl − 2)**, die Höhe **6 + 2 × (Spielerzahl − 2)**. Die Gewinnlänge bleibt stets vier.

© 2026 Julian Krauß | JK-PT-2026-051-RA
DESIGN: JK-CD-DE-2026-003 / REV. 3.2

03 / MUSS-KRITERIEN

Funktionale Anforderungen I

Die IDs FR-01 bis FR-06 beschreiben Einstieg, Spielerverwaltung und Grundlogik.

FR-01	**Hauptmenü** Nach dem Start zeigt die Konsole die auswählbaren Funktionen 0-6. **Abnahme:** Eingabe 0 beendet; gültige Menüpunkte lösen zugeordnete Aktionen aus.	AB51 S.2	P01
FR-02	**Spielerzahl** Vor Spielbeginn sind genau 2, 3 oder 4 Mitspielende auswählbar. **Abnahme:** Andere Zahlen werden abgefangen und erneut abgefragt.	AB51 S.2	P01
FR-03	**Spielernamen** Jeder aktive Spieler erhält einen nichtleeren Namen. **Abnahme:** Leere Eingabe wird zurückgewiesen; kein Start mit fehlendem Namen.	AB51 S.2/8	P01
FR-04	**Skalierbares Feld** 7×6 (2), 9×8 (3), 11×10 (4); Dimensionen werden zentral ermittelt. **Abnahme:** Alle drei Spielerzahlen erzeugen exakt passende Raster.	AB51 S.3	P02
FR-05	**Steinplatzierung** Ein Zug wählt eine vorhandene Spalte; das Symbol sinkt in das unterste freie Feld. **Abnahme:** Kein Schweben; belegte Felder bleiben unverändert.	AB51 S.2/8	P03
FR-06	**Zugreihenfolge** Die Spieler sind reihum aktiv; nach gültigem Zug folgt der nächste. **Abnahme:** Bei ungültigem Zug bleibt derselbe Spieler am Zug.	AB51 S.2/8	P02

© 2026 Julian Krauß | JK-PT-2026-051-RA
DESIGN: JK-CD-DE-2026-003 / REV. 3.2

04 / MUSS-KRITERIEN

Funktionale Anforderungen II

Die IDs FR-07 bis FR-12 sichern Regelwerk, Fehlerbehandlung und Datenspeicherung.

FR-07	**Gewinnerkennung** Vier zusammenhängende eigene Symbole horizontal, vertikal oder in einer der beiden Diagonalen gewinnen.	AB51 S.3/8	P05
	Abnahme: Alle vier Richtungsfälle werden zuverlässig erkannt.		
FR-08	**Unentschieden** Ist das gesamte Spielfeld voll und niemand hat gewonnen, endet die Partie unentschieden.	AB51 S.2/8	P04
	Abnahme: Remis wird ohne Highscore-Zuwachs ausgegeben.		
FR-09	**Ungültige Eingaben** Nichtnumerische, falsche oder volle Spalten sind kein gültiger Spielzug.	AB51 S.2/8	P03
	Abnahme: Kein Absturz, keine Zustandsänderung; neue Eingabe möglich.		
FR-10	**Spiel speichern** Eine Partie wird mit Formatversion, Namen, Spielerzahl, Symbolen, Spielstatus und Zellinhalt gespeichert.	AB51 S.4	P06
	Abnahme: Datei ist lesbar und für Wiederherstellung vollständig.		
FR-11	**Spiel laden** Lokale Datei wird validiert und die Partie bei gültigem Inhalt rekonstruiert.	AB51 S.4/8	P07
	Abnahme: Spielbrett, aktueller Spieler und Namen stimmen exakt überein.		
FR-12	**Highscore** Siege pro Spielername dauerhaft speichern und absteigend sortiert anzeigen.	AB51 S.4/8	P08
	Abnahme: Jeder Sieg +1; Gleichstände stabil/alphabetisch sortieren.		

© 2026 Julian Krauß | JK-PT-2026-051-RA

05 / MUSS-KRITERIEN

Funktionale Anforderungen III

Programmzustände, Qualitätssicherung und geforderte Abgabe-Unterlagen.

FR-13	**Partieabschluss** Nach Sieg/Remis wird das Ergebnis angezeigt und zum Hauptmenü zurückgekehrt.	AB51 S.2	P04
	Abnahme: Kein weiterer Stein nach Spielende; neue Partie wählbar.		
FR-14	**Dateifehler behandeln** Fehlende, unlesbare oder fehlerhafte Spielstandsdateien werden sicher behandelt.	AB51 S.4/8	P07
	Abnahme: Verständliche Meldung; aktiver Zustand wird nicht beschädigt.		
FR-15	**GitHub-Historie** Sourcecode und Doku liegen versioniert in Repository-Struktur mit Commits vor.	AB51 S.2/7	DOC
	Abnahme: Repo, Commits und ausgewählter Code in der Live-Demo vorzeigbar.		
FR-16	**KI-Prompts dokumentieren** Verwendete KI-Prompts sind nachvollziehbar gesammelt und die Nutzung erklärbar.	AB51 S.2/10	DOC
	Abnahme: Prompts und Prüf-/Änderungsschritte einsehbar.		
FR-17	**Handout und Präsentation** Es gibt ein Handout und eine ca. 30-minütige Vorstellung mit Live-Demo.	AB51 S.9-12	DOC
	Abnahme: Lehrkraft sieht Architektur, Bedienung, Testfälle und Repository.		
FR-18	**Quellcode verstehen** Wichtige Methoden, Klassenaufgaben und Algorithmen werden selbstständig erläutert.	AB51 S.13	DOC
	Abnahme: Entwickler erklärt Gewinnerkennung, Dateiprüfung und Zustandswechsel.		

© 2026 Julian Krauß | JK-PT-2026-051-RA

06 / ERWEITERUNG

Geplante Sonderfunktionen

Diese Funktionen verbessern die Eigenständigkeit der Gym Edition, sind aber keine zusätzlichen Muss-Kriterien der Schule.

EX-01	**Kurzhantel-Symbole** Rote/blaue bzw. eindeutige ASCII-Hanteln kennzeichnen die Spielfiguren. **Abnahme:** Symbole lassen sich pro Spieler eindeutig unterscheiden.	Projektidee	UI
EX-02	**Zugvorschau** Befehl V n zeigt die mögliche Zielzelle, ohne einen Stein zu platzieren. **Abnahme:** Brett, Spieler und Zughistorie bleiben identisch.	Projektidee	P10
EX-03	**Rückgängig** Befehl U nimmt den letzten gültigen Zug zurück; Regelung für Laden/Spielende festlegen. **Abnahme:** Nach Undo ist vorheriger Zustand exakt rekonstruiert.	Projektidee	P11
EX-04	**Schnellspeichern** Befehl S speichert direkt während der Partie. **Abnahme:** Speicherfehler werden erklärt; Partie bleibt aktiv.	Projektidee	P09
EX-05	**Ingame-Hilfe** Befehl H erklärt Steuerung und Ziel, ohne einen Zug zu verbrauchen. **Abnahme:** Hilfe sichtbar; Spielzustand unverändert.	Projektidee	P09
EX-06	**Match Insights** Befehl I zeigt Züge, Restfelder und Füllgrad aus dem Spielfeldzustand. **Abnahme:** Summe besetzt + frei = alle Zellen.	Projektidee	P12
EX-07	**Fallanimation / 2D-UI** Optionale Konsolenanimation; separat denkbare grafische Oberfläche nach den Pflichtfunktionen. **Abnahme:** Konsole bleibt auch ohne Animation vollständig spielbar.	Projektidee	UI

© 2026 Julian Krauß | JK-PT-2026-051-RA
DESIGN: JK-CD-DE-2026-003 / REV. 3.2

07 / DARSTELLUNG

Gaming-UI und Konsolenbetrieb

Visuelles Konzept und Programmverhalten sind technisch zu unterscheiden.

01

Bedienung

Spielerzahl und Name → Spalte → bestätigter Spielzug.

02

Logik

Regelvalidierung, Position, Gewinn / Remis, Zugwechsel.

03

Ansicht

Spielfeld rendern; Kurzhantel-Symbole statt klassischer Scheiben.

Verbindliche Konsolenoberfläche

- **Menü:** Textbasierte Auswahl, numerische Eingaben, klar gekennzeichneter aktiver Spieler und belegte Spalten.
- **Kurzhantel-Visualisierung:** Die gezeichneten Hanteln sind Symbole; das Raster speichert weiterhin nur eindeutige Spielerzeichen.
- **Fallbewegung:** Als freiwilliger Effekt darf eine Animation die Ausgabe verändern, nicht den Spielzustand.

Grafische Oberfläche: bewusste spätere Option

- **UI-Mockup:** Die gezeigten Gaming-Bilder sind Designstudien ohne echte Buttons oder 3D-Engine im C++17-Lernpaket.
- **Technischer Ausbau:** Für ein klickbares 2D-Spiel wären Grafikbibliothek, Renderer, Eingabeereignisse und Rendering-Tests erforderlich.

ENTSCHEIDUNG ZUR BEWERTUNG

Eine belastbar funktionierende Konsole mit gutem Code und Tests hat Vorrang. Der Gaming-Look darf die in Aufgabenblatt 51 geforderten Funktionen nicht verdrängen.

© 2026 Julian Krauß | JK-PT-2026-051-RA
DESIGN: JK-CD-DE-2026-003 / REV. 3.2

08 / QUALITÄT

Nichtfunktionale Anforderungen

Messbare Qualitätskriterien ergänzen das fachliche Verhalten des Programms.

ID	Kriterium	Überprüfbare Festlegung	Nachweis
NFR-01	Stabilität	Normale und ungültige Benutzereingaben verursachen keinen Absturz.	TC-09, TC-14
NFR-02	Portabilität	Code kompiliert mit einem C++17-Compiler; keine zwingende GUI-Abhängigkeit.	Build-Nachweis
NFR-03	Konsolen-Lesbarkeit	Felder und Spaltennummern sind für 7/9/11 Spalten klar lesbar.	Sichtprüfung
NFR-04	Eingabekontrolle	Grenzen 2-4 Spieler und 1...Spaltenzahl werden systematisch geprüft.	TC-02, TC-05
NFR-05	Datenintegrität	Fehler beim Laden verändern keine gültige laufende Partie.	TC-14
NFR-06	Wartbarkeit	Player, Board und Game haben abgegrenzte Verantwortlichkeiten.	Code-Review
NFR-07	Nachvollziehbarkeit	Jeder Meilenstein hat einen Commit und kurzen Testnachweis.	Repo-Prüfung
NFR-08	Determinismus	Gleicher gültiger Spielzustand führt zum gleichen Ergebnis.	Unit-Tests
NFR-09	Bedienbarkeit	Fehlermeldung erklärt Ursache und erlaubt erneute Eingabe.	Sichtprüfung
NFR-10	Datenschutz	Keine Netzwerkübertragung oder unnötige personenbezogene Daten.	Datenfluss-Prüfung

DEFINITION OF DONE

Ein Feature gilt erst dann als fertig, wenn es kompiliert, den vorgesehenen Test besteht, dokumentiert und mit verständlichem Commit im Repository hinterlegt ist.

© 2026 Julian Krauß | JK-PT-2026-051-RA

09 / OOP-ARCHITEKTUR

Klassen und Verantwortlichkeiten

Die Architektur trennt Datenspeicherung, Spiellogik und Eingabesteuerung so weit wie für ein Schulprojekt sinnvoll.

Player	Name und Spielsymbol einer Person.
BEISPIELMETHODEN	getName(), getSymbol()

Board	Dimensionen, Raster, Platzierung, Vorschau und Gewinnprüfung.
BEISPIELMETHODEN	drop(), hasWon(), isFull(), undo()

Game	Menü, Regeln zur Partie, Reihenfolge, Speichern/Laden und Highscore.
BEISPIELMETHODEN	run(), play(), saveGame(), loadGame()

Datenhaltung und Zustandswechsel

- **Player / Board:** Player verwaltet pro Person Name/Symbol. Board besitzt Zellen und kennt die Gewinnlogik.
- **Game:** Game hält aktiven Spieler, Partie-Status und Zug-Historie und delegiert Brettoperationen an Board.
- **Rendering:** ASCII-Kurzhanteln sind eine Anzeigeentscheidung; sie dürfen die Board-Datenstruktur nicht verändern.

BEGRÜNDUNG

Eine klare Trennung verhindert doppelte Spiellogik. Die Gewinnprüfung kann unabhängig vom Menü getestet werden, auch wenn die Zahl der Mitspielenden wächst.

© 2026 Julian Krauß | JK-PT-2026-051-RA

10 / PERSISTENZ

Speichern, Laden und Highscore

Dateiformate werden versioniert; Eingaben aus Dateien gelten niemals automatisch als korrekt.

Pflichtangaben für einen Spielstand

Feld	Anforderung / Prüfschritt
Formatversion	Eine erkennbare Version steuert später mögliche Formatänderungen.
Spielerinformationen	Spielerzahl 2-4, pro Person gültiger Name und eindeutiges Spielsymbol.
Spielzustand	Aktiver Spieler, Feldbreite/-höhe und tatsächlicher Inhalt aller Rasterzellen.
Plausibilität	Dimensionen zur Spielerzahl passend; keine unzulässigen Zeichen oder fehlende Zeilen.
Optional	Zughistorie für Undo nur speichern, wenn die Funktion nach Laden unterstützt werden soll.

Transaktionsprinzip beim Laden

01 Datei öffnen und vollständig lesen

02 Version, Zahlen, Namen und Spieldimensionen prüfen

03 Raster und aktuellen Spieler auf Plausibilität prüfen

04 Erst dann den laufenden Zustand austauschen

Highscore

- **Persistenz:** Siege zu Spielernamen in einer lokalen Datei ablegen und beim Programmstart erneut einlesen.
- **Aktualisierung:** Nur bei einem tatsächlich festgestellten Sieg einmal um eins erhöhen. Remis und abgebrochene Spiele zählen nicht.
- **Sortierung:** Absteigend nach Siegen; gleiche Siegzahl alphabetisch oder sonst nachvollziehbar stabil sortiert.

© 2026 Julian Krauß | JK-PT-2026-051-RA
DESIGN: JK-CD-DE-2026-003 / REV. 3.2

11 / PRÜFKONZEPT

Testfälle: Spiellogik

Alle erwarteten Ergebnisse sind Soll-Kriterien; ein bestandenes Testergebnis darf erst nach praktischer Ausführung eingetragen werden.

Test	Situation	Erwartetes Ergebnis	Anf.
TC-01	Hauptmenü	0 beendet, 1 startet, 6 zeigt Rangliste.	FR-01
TC-02	Spielerzahl	1 und 5 abweisen; 2, 3 und 4 annehmen.	FR-02
TC-03	Name fehlt	Leerer Name löst neue Nachfrage aus.	FR-03
TC-04	Rastergrößen	2→7×6, 3→9×8, 4→11×10.	FR-04
TC-05	Ungültige Spalte	0, zu groß und volle Spalte bleiben wirkungslos.	FR-05/09
TC-06	Zugwechsel	Bei gültigem Zug rotiert aktueller Spieler.	FR-06
TC-07	Horizontaler Gewinn	Vier in einer Reihe waagerecht → Sieg.	FR-07
TC-08	Vertikaler Gewinn	Vier in einer Spalte → Sieg.	FR-07
TC-09	Diagonalgewinn / und \	Beide Diagonalrichtungen werden erkannt.	FR-07
TC-10	Volles Feld	Ohne Viererreihe wird Remis gemeldet.	FR-08

WIE DOKUMENTIEREN?

Pro Test Datum, Version/Commit, konkrete Eingaben, Soll, Ist und Status **bestanden / nicht bestanden** erfassen. In diesem Plan werden keine Prüfergebnisse vorweggenommen.

12 / PRÜFKONZEPT

Testfälle: Dateien und Extras

Die freiwilligen Sonderfunktionen werden separat geprüft, damit Fehler dort die Kernabnahme nicht gefährden.

Test	Situation	Erwartetes Ergebnis	Anf.
TC-11	Speichern und Laden	Partie nach Neustart identisch wiederhergestellt.	FR-10/11
TC-12	Beschädigte Datei	Fehlermeldung, kein Absturz, kein Datenverlust.	FR-14
TC-13	Highscore	Genau ein Sieg zusätzlich und korrekte Sortierung.	FR-12
TC-14	Spielende	Gewinnerausgabe; Rückkehr zum Menü.	FR-13
TC-15	GitHub	Commits und Struktur in Repository auffindbar.	FR-15
TC-16	Prompts / Handout	Doku und Live-Demo vollständig nachgewiesen.	FR-16-18
TC-17	Vorschau	Keine Änderung an Zellen, Spieler oder Historie.	EX-02
TC-18	Rückgängig	Genau der letzte gültige Zug verschwindet.	EX-03
TC-19	Schnellspeichern	Spiel läuft nach Speicherung weiter.	EX-04
TC-20	Hilfe / Statistik	Keine Änderung der Partie; Zahlen korrekt.	EX-05/06
TC-21	Fallanimation	Abschaltbar, ohne Einfluss auf Spiellogik.	EX-07

MINIMALER PRÜFNACHWEIS

Mindestens die von Aufgabenblatt 51 genannten Tests zeigen. Für Zusatzfunktionen ist ein kurzer eigener Test pro Befehlsart sinnvoll, aber erst nach stabiler Grundversion.

© 2026 Julian Krauß | JK-PT-2026-051-RA

13 / TRACEABILITY

Rückverfolgbarkeit: Pflichtkern

Die Verbindung zwischen Anforderung, PAP und Testfall verhindert fehlende oder nur scheinbar abgedeckte Funktionen.

Anf.	Thema	PAP	Testfall
FR-01	Menü	P01	TC-01
FR-02	Spielerzahl	P01	TC-02
FR-03	Namen	P01	TC-03
FR-04	Feldgröße	P02	TC-04
FR-05	Steinplatzierung	P03	TC-05
FR-06	Zugwechsel	P02	TC-06
FR-07	Gewinn	P05	TC-07 bis TC-09
FR-08	Remis	P04	TC-10
FR-09	Fehleingabe	P03	TC-05
FR-10	Speichern	P06	TC-11
FR-11	Laden	P07	TC-11
FR-12	Highscore	P08	TC-13

ABDECKUNGSREGEL

Für jedes Pflichtmerkmal muss mindestens **eine technische Realisierung** und **ein objektiver Test** vorhanden sein. Ein PAP allein gilt nicht als Funktionsnachweis.

© 2026 Julian Krauß | JK-PT-2026-051-RA

14 / TRACEABILITY

Rückverfolgbarkeit: Projektabschluss

Dokumentationspflichten und Erweiterungen benötigen neben Code auch sichtbare Nachweise.

Restliche Pflichtanforderungen

Anf.	Thema	Prüf-/Dokunachweis
FR-13	Partieabschluss	P04 · TC-14 · Live-Demo
FR-14	Fehlerhafte Datei	P07 · TC-12 · Protokoll einer ungültigen Datei
FR-15	GitHub-Versionierung	TC-15 · Repository, Commits und Release
FR-16	KI-Prompt-Sammlung	TC-16 · Prompt-Liste mit Reflexion
FR-17	Handout / Präsentation	TC-16 · Handout, Folien, Generalprobe
FR-18	Verständnis	Fachgespräch · ausgewählte Quellcode-Erklärung

Optionale Sonderfunktionen

Extra	PAP / Technik	Test
EX-01	Anzeigerendering / ASCII	Sichtprüfung
EX-02	P10 · Zugvorschau	TC-17
EX-03	P11 · Rückgängig	TC-18
EX-04	P09 · Schnellspeichern	TC-19
EX-05	P09 · Hilfe	TC-20
EX-06	P12 · Insights	TC-20
EX-07	Optionaler Animationseffekt	TC-21

© 2026 Julian Krauß | JK-PT-2026-051-RA
DESIGN: JK-CD-DE-2026-003 / REV. 3.2

15 / PLANUNG

Priorisierung und Meilensteine

Terminplan gemäß Aufgabenblatt 51; Zusatzfunktionen sind nachrangig und dürfen Pflichttermine nicht gefährden.

MoSCoW-Priorisierung

Priorität	Umfang	Abnahmeregel
MUST	FR-01 bis FR-18, NFR-01 bis NFR-10	Ohne diese Punkte keine fachlich vollständige Schulabgabe.
SHOULD	EX-01 / EX-05; klare Kurzhantel-ASCII-Ausgabe, Hilfe	Erst nach dem funktionalen Kern.
COULD	EX-02 / EX-03 / EX-04 / EX-06 / EX-07	Vorschau, Undo, Quick Save, Insights, Animation.
WON'T (aktuell)	3D-Game-Engine, Online-Modus, KI-Gegner	Nicht Bestandteil der ersten Abnahme.

Verbindliche Meilensteine aus dem Aufgabenblatt

Termin	Erwartetes Ergebnis
29.10.2026	Anforderungen, Klassenentwurf, GitHub-Repository und Menü-Prototyp.
26.11.2026	Zwei-Spieler-Grundversion mit validierten Zügen.
17.12.2026	Gewinn, Remis und Kernregelwerk getestet.
28.01.2027	2-4 Spieler, Dateifunktionen, Highscore; Feature Freeze.
11.02.2027	Generalprobe der Live-Demo.
18.02.2027	Vorführung, Dokumentation, Handout und Quellcode erläutern.

ENTWICKLUNGSENTSCHEIDUNG

Ein Extras-Feature darf gestrichen oder verschoben werden, wenn sonst die stabile Pflichtfunktion, Dokumentationsqualität oder Vorbereitung auf die Präsentation leidet.

© 2026 Julian Krauß | JK-PT-2026-051-RA

16 / ENTSCHEIDUNGEN

Risiken und offene Punkte

Risiken sind konkrete technische Unsicherheiten. Sie müssen im Projektverlauf entschieden und versioniert werden.

Risikobetrachtung und Maßnahmen

ID	Risiko	Schwere	Gegenmaßnahme
R1	Zusatzfunktionen kosten zu viel Zeit	Hoch	Pflichtfunktionen zuerst; Termin für Feature Freeze.
R2	Dateifehler beschädigt Partie	Hoch	Vor dem Überschreiben vollständig in temporären Zustand laden.
R3	Fehler in diagonaler Gewinnprüfung	Hoch	Tests für beide Diagonalen und Randpositionen.
R4	ASCII-/ANSI-Effekte je Konsole verschieden	Mittel	Animation abschaltbar; reine Textansicht als Rückfall.
R5	Design und C++-Umsetzung werden verwechselt	Mittel	3D-Visualisierung als Mockup kennzeichnen; reale Konsole zeigen.
R6	Abgabe ohne Commits/Prompts	Mittel	Wöchentliche GitHub-Pflege und Prompt-Protokoll.

Offene Entscheidungen für die Umsetzung

- **Undo nach dem Laden:** Ist die Zughistorie Teil des Speicherformats? Falls nein, Undo für geladene Spiele deaktivieren.
- **Sonderbefehle:** Wie werden Befehl S, V n, H und U von Spaltenzahlen sicher unterschieden?
- **Gleichnamige Spieler:** Werden gleiche Namen abgewiesen, oder wie würden Highscores unterschieden? Empfehlung: Namen je Partie eindeutig halten.
- **Grafikbibliothek:** Nur nach vollständiger Pflichterfüllung entscheiden, ob eine echte 2D-UI noch sinnvoll ist.

© 2026 Julian Krauß | JK-PT-2026-051-RA

17 / ABSCHLUSS

Abnahmecheckliste und Quellen

Diese Seite dient als Checkliste für den letzten Funktionsnachweis vor der Präsentation.

- [ ] Programm kompiliert mit C++17 ohne Compilerfehler.
- [ ] 2, 3 und 4 Spieler erzeugen die korrekten Spielfeldgrößen.
- [ ] Gewinn und Remis werden in allen relevanten Fällen erkannt.
- [ ] Ungültige Eingaben ändern weder Brett noch Spieler.
- [ ] Spielstand lässt sich speichern und nach Neustart korrekt laden.
- [ ] Fehlende / beschädigte Dateien werden sicher abgefangen.
- [ ] Highscore bleibt dauerhaft, zählt nur Siege und sortiert korrekt.
- [ ] GitHub-Commits, Quellcode, Handout und KI-Prompts sind nachweisbar.
- [ ] Live-Demo und Fachfragen zur Klassenstruktur sind geübt.
- [ ] Optionale Gym-Funktionen sind separat getestet oder ausdrücklich als geplant markiert.

Quellen / eindeutige Einordnung

- **[Q1] Aufgabenblatt 51,** Projektaufgabe Programmiertechnik 2026/2027, 13 Seiten; insbesondere S. 2-4, 6-8 und 12-13.
- **[Q2] Projektdokumentation,** „Vier gewinnt: Gym Edition“ mit interaktivem Inhaltsverzeichnis, Programmierleitfaden und P01-P12, Stand 22.10.2026.
- **[Q3] C++17-Lernprojekt,** lokales ZIP mit Player.h, Board.h/.cpp, Game.h/.cpp, main.cpp, Testdateien (Arbeitsstand).
- **[Q4] Corporate Design Guide,** JK-CD-DE-2026-003, Revision 3.2 / Release 4.6.3.

STATUS DIESER ANALYSE

Dokumentierte Planung, keine Abnahmebescheinigung. Anforderungsdefinitionen und Solltests sind ausgearbeitet. Reale Testergebnisse, Unterschriften und die finale Implementierung sind im Entwicklungsverlauf zu ergänzen.